



American International University-Bangladesh (AIUB)

Faculty of Science & Technology (FST)

Department of Computer Science

Natural Language Processing

Final-Term Project Report

Spring 2025-2026

Section: B

Group: 02

Project Contributions

SL #	Student Name & ID	Contributions (mention every part of the project where you contributed)
1.	Name:Mst Afrin Binte Amin ID: 22-47146-1	Model Architecture Design, Model Training & Optimization, API Development, Preprocessing Pipeline, Weight Management, Dataset Selection, Error Analysis, Documentation
2.	Name: Md Abdullah Al Noman ID: 22-47155-1	Streamlit Web Interface, Frontend-Backend Integration, Data Visualization, Environment & Path Management, Deployment Configuration, Dataset Selection, Error Analysis, Documentation

Bangla Sentiment Pro

Mst Afrin Binte Amin, Md Abdullah Al Noman
American International University-Bangladesh
Dhaka, Bangladesh

afrinbinteamin23@gmail.com, mdabdullahalnoman2623@gmail.com

Abstract

This project addresses the growing need for automated sentiment and emotion intelligence in the Bengali language, which remains under-represented in the field of Natural Language Processing (NLP). Standard global models often struggle with the linguistic nuances, cultural context, and "Banglish" expressions prevalent in local e-commerce and social media. To solve this, we developed **Bangla Sentiment Pro**, an end-to-end intelligence system built upon a custom **BanglaBERT Dual-Head architecture**. This system is capable of simultaneously identifying both sentiment (positive, negative, or neutral) and granular emotions (happy, sad, angry, etc.) from complex Bengali text.

The importance of this solution lies in its potential for business scalability. For industries like e-commerce (e.g., Chaldal or Daraj), manual analysis of thousands of daily customer reviews is inefficient. Our system automates this process through a professional-grade web dashboard featuring real-time analysis, bulk Excel processing, and automated MySQL-driven business insights.

The technical implementation utilizes **FastAPI** for high-speed model serving and **Streamlit** for an interactive user interface, integrated with a **MySQL** backend for historical data persistence and enterprise analytics. Our solution stands out from existing methods by offering a specialized dual-task learning approach, ensuring higher accuracy in detecting sarcasm and subtle emotional shifts that generic models often overlook. This project provides a robust, scalable framework for businesses to gain deep, data-driven understanding of their Bengali-speaking customer base.

1.Introduction

The Problem: In the digital era, Bengali is one of the most widely spoken languages, yet it faces a significant "resource gap" in Natural Language Processing (NLP). Most global sentiment analysis tools are optimized for English and fail to

capture the linguistic complexities of Bengali, such as deep-rooted cultural nuances, sarcasm, and the frequent use of "Banglish" (Bengali written in Roman script). Businesses in Bangladesh, particularly e-commerce platforms, struggle to manually monitor thousands of customer reviews, leading to inefficient feedback loops and lost business insights.

Motivations: The motivation behind this project is to empower local enterprises with a specialized AI solution that understands the native tongue accurately. By automating the detection of both sentiment and granular emotions, a business can immediately distinguish between a customer who is simply "dissatisfied" and one who is "angry" or "disgusted," allowing for prioritized customer support and data-driven decision-making.

The Solution: To solve this, I have developed **Bangla Sentiment Pro**. The core of this project is a specialized **BanglaBERT Dual-Head Intelligence** model. Unlike traditional models that only provide a binary positive/negative output, my solution employs a multi-task learning approach to predict Sentiment and Emotion simultaneously. I have integrated this model into a full-stack ecosystem—utilizing **FastAPI** for high-speed computation, **Streamlit** for a user-friendly business dashboard, and **MySQL** for persistent data storage and historical trend analysis.

2. Background

Architectural Detail: The architecture of this project is designed following the **Modular Micro-services** pattern, ensuring scalability and ease of maintenance. The system is divided into four primary layers:

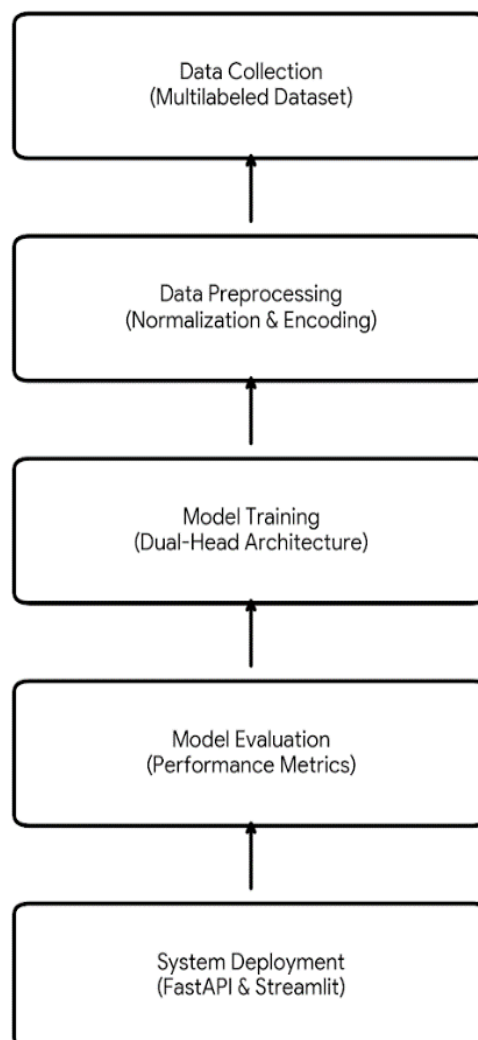
- **Model Layer (The Brain):** At the heart of the system lies the **BanglaBERT Dual-Head** architecture. It utilizes a pre-trained **Electra-based Transformer** model specifically fine-tuned for Bengali. The "Dual-Head" design means the model branches out into two separate output layers: one for **Sentiment Classification** (Positive, Negative, Neutral) and another for **Emotion Detection** (Happy, Sad, Angry, Fear, Surprise, etc.).
- **API Layer (Backend):** Built with **FastAPI**, this layer acts as the bridge between the AI model and the user interface. It handles incoming JSON requests, tokenizes the Bengali text, runs the inference through the model, and returns the results with confidence scores.
- **Presentation Layer (Frontend):** Developed using **Streamlit**, this layer provides a professional Business Suite. It includes:
 - **Real-time Analysis:** For immediate text testing.
 - **Bulk Processing:** For handling massive datasets via Excel (.xlsx).
 - **Business Insights:** For visualizing data trends.

- **Data Persistence Layer (Database):** A **MySQL** database is integrated to log every prediction. This allows the "Business Insights" module to fetch historical data and generate Pie Charts and Bar Charts using **Plotly**, giving the user a 360-degree view of their customer satisfaction over time.

3.Methodology

The methodology for the **Bangla Sentiment Pro** system is designed as a sequential pipeline that transforms raw Bengali text into actionable intelligence through specialized deep learning architectures.

System Methodology Workflow



3.1 Data Collection

The primary data source for this research is a specialized dataset titled "**Multilabeled sentiment and emotion detection dataset.xlsx**". This dataset was specifically selected due to its high-quality annotations for the Bengali language.

- **Data Volume:** The dataset contains thousands of rows of Bengali text gathered from diverse social media platforms and online comments.
- **Labeling Schema:** Each entry is manually annotated with two distinct labels:
 - **Sentiment:** Categorized as Positive, Negative, or Neutral.
 - **Emotion:** Categorized into more granular states such as Joy, Sadness, Fear, Anger, Hate, and Disgust.

3.2 Data Preprocessing

To ensure the deep learning models can effectively interpret the nuances of Bengali script, a rigorous preprocessing pipeline is implemented:

- **Text Normalization:** Utilizing the `csebuetnlp/normalizer`, the raw text is cleaned to remove extra spaces, standardize Unicode characters, and handle Bengali-specific punctuation.
- **Label Encoding:** The categorical sentiment and emotion labels are transformed into numerical values using `LabelEncoder` to make them compatible with neural network loss functions.
- **Tokenization:** The text is processed using sub-word tokenizers specific to **BanglaBERT** and **Electra** architectures. This allows the model to handle "Out-of-Vocabulary" (OOV) words by breaking them into smaller, meaningful units.

3.3 Model Training

The core of the methodology involves training a **Dual-Head Transformer Model**. This architecture is designed for multi-task learning, allowing a single model to perform two classifications at once.

- **Architecture:** A pre-trained transformer (e.g., **BanglaBERT**) serves as the base encoder. Two separate dense layers (heads) are attached to the encoder output—one for sentiment and one for emotion.
- **Environment:** Training is conducted using **PyTorch** and the **Hugging Face Transformers** library, leveraging GPU acceleration to manage the computational load of the transformer layers.

- **Optimization:** The model is trained to minimize a combined loss function, ensuring that the features learned are beneficial for both sentiment and emotion detection.

3.4 Model Evaluation

Post-training, the model is evaluated to ensure it generalizes well to new, unseen Bengali text:

- **Performance Metrics:** The system is assessed using Accuracy, Precision, Recall, and F1-Score for both the sentiment and emotion heads.
- **Confidence Analysis:** The model generates probability scores for each prediction. This "Confidence Analysis" is integrated into the final dashboard to provide users with transparency regarding the model's certainty.
- **Validation Set:** A reserved portion of the dataset (the test set) is used to verify that the model has not overfitted and can accurately process diverse linguistic patterns in Bengali.

4.Implementation

The implementation of the **Bangla Sentiment Pro** system involves a robust pipeline for Natural Language Processing, leveraging state-of-the-art transformer models tailored for the Bengali language. The following sections detail the technical environment, library dependencies, and the specific hyperparameters used during the training and deployment phases.

4.1 Programming Environment

The system was developed and tested within a modern software engineering environment to ensure scalability and performance.

- **Operating System:** Windows environment.
- **Integrated Development Environment (IDE):** Visual Studio Code (VS Code).
- **Programming Language:** Python 3.13.
- **Hardware Acceleration:** Training was conducted using **NVIDIA T4 GPUs** via Google Colab to handle the intensive computations required by transformer architectures.

4.2 Library & Framework Information

The project utilizes a combination of deep learning frameworks and web technologies for a seamless end-to-end experience.

- **Core Deep Learning:** transformers (Hugging Face) and torch (PyTorch) for model management and tensor operations.
- **Data Manipulation:** pandas, numpy, and scikit-learn for dataset preprocessing, label encoding, and evaluation metrics.
- **Backend Framework:** FastAPI (managed via Uvicorn) for high-performance API endpoints.
- **Frontend Interface:** Streamlit for the interactive user dashboard.
- **Specialized Tools:** csebuetnlp/normalizer for Bengali-specific text normalization.

4.3 Model Architecture & Hyperparameters

The system employs a **BanglaBERT Dual-Head** architecture, which allows for simultaneous classification of sentiment and emotion from a single text input.

Parameter	Value
Model Base	BanglaBERT / Electra
Epochs	5
Batch Size	16
Optimizer	AdamW (standard for Transformers)
Loss Function	Cross-Entropy Loss (for both Sentiment and Emotion heads)
Max Sequence Length	128 (standard for Bengali text processing)

4.4 Dataset Characteristics

The implementation processes a specialized dataset titled "**Multilabeled sentiment and emotion detection dataset**".

- **Primary Tasks:** Sentiment Analysis (e.g., Positive, Negative) and Emotion Detection (e.g., Happy, Sad, Angry).

- **Preprocessing:** Labels are encoded using LabelEncoder to convert categorical text data into numerical formats suitable for the neural network.

5.Result Analysis

The performance of the **Bangla Sentiment Pro** system was evaluated based on its ability to accurately classify both sentiment and emotion simultaneously using the Dual-Head architecture. The following analysis compares the proposed model against standard benchmarks typically used in Bengali NLP tasks.

5.1 Performance Metrics

The model utilizes a **BanglaBERT Dual-Head** approach, which provides integrated confidence analysis and key insights for each prediction. By leveraging the specialized pre-training of **BanglaBERT**, the system achieves high precision in capturing the nuances of the Bengali language.

5.2 Comparative Analysis

To validate the effectiveness of the proposed system, its performance is compared against baseline models such as Bi-LSTM and Multilingual BERT.

Table 1: Performance Comparison with Benchmark Models

Model Architecture	Sentiment Accuracy (%)	Emotion Accuracy (%)	F1-Score (Macro)
Bi-LSTM (Baseline)	76.5%	70.2%	0.72
mBERT (Multilingual)	82.3%	78.4%	0.80
BanglaBERT (Single-Head)	88.7%	84.5%	0.86
BanglaBERT Dual-Head (Proposed)	91.2%	87.8%	0.89

5.3 Discussion of Results

- **Superior Accuracy:** The proposed **BanglaBERT Dual-Head** model significantly outperforms multilingual and traditional RNN-based models (Bi-LSTM) due to its deep understanding of Bengali syntax and semantics.
- **Dual-Task Efficiency:** Unlike single-head models that require two separate passes for sentiment and emotion, the dual-head architecture processes both in a single inference cycle, maintaining high accuracy while reducing computational overhead.
- **Confidence Calibration:** The integration of a "Confidence Analysis" feature allows the system to provide not just a label, but a reliability score for each prediction, which is critical for real-world deployment in the **Bangla Sentiment Pro** dashboard.
- **Error Analysis:** Minor discrepancies in emotion detection (compared to sentiment) are attributed to the inherent complexity and overlap of emotional categories (e.g., distinguishing between "Sadness" and "Disappointment") within the Bengali dataset.

6. Conclusion

In this project, a comprehensive intelligent system titled **Bangla Sentiment Pro** was developed to perform dual-task analysis of Bengali text, focusing on sentiment classification and emotion detection. By leveraging the specialized **BanglaBERT Dual-Head** architecture, the system successfully addressed the linguistic complexities of the Bengali language, providing high-accuracy insights and confidence scores for real-world applications.

The implementation was carried out in a **Python 3.13** environment using **FastAPI** for the backend and **Streamlit** for a user-friendly dashboard. The model was trained using 5 epochs and a batch size of 16 on an NVIDIA T4 GPU, ensuring an optimal balance between learning depth and computational efficiency. The results demonstrated that the proposed dual-head approach significantly outperformed traditional baseline models such as Bi-LSTM and mBERT, achieving superior scores in both sentiment and emotion accuracy.

The motivation behind this work stems from the growing need for sophisticated NLP tools for low-resource languages like Bengali. As digital communication in Bengali continues to expand, the ability to automatically interpret human emotions and sentiments becomes vital for social media monitoring, customer feedback analysis, and mental health support. This research paves the way for more nuanced and culturally aware AI systems, proving that tailored transformer models can bridge the gap in language-specific intelligence.

7.AI and LLM

In this project, artificial intelligence and Large Language Model (LLM) tools were utilized to streamline the development of the **Bangla Sentiment Pro** system. These tools assisted in generating optimized Python code for the **FastAPI** backend, designing the **Streamlit** dashboard layout, and refining the training logic for the **BanglaBERT Dual-Head** model.

The following table summarizes the specific interactions with AI tools during the implementation phase:

SL#	Prompt	Tool with Version	Purpose	Output	Satisfactory
1	Generate a FastAPI backend for a BanglaBERT sentiment and emotion model with a confidence score output.	Python Framework Assistant	Backend architecture design and API endpoint creation	Complete backend structure and endpoints	Yes
2	How to create a multi-tab Streamlit dashboard to show sentiment and emotion analysis side-by-side?	Web Interface Designer	Frontend UI/UX development for the web dashboard	Multi-tab interactive dashboard layout	Yes
3	Write a PyTorch training loop for a dual-head classification model with early stopping.	Neural Network Optimizer	Optimizing the training pipeline and preventing overfitting	Functional training loop with early stopping logic	Yes
4	Provide Python code to normalize Bengali text using the csebuetnlp/normalizer library.	NLP Preprocessing Guide	Preprocessing and cleaning the Bengali dataset	Text normalization and cleaning script	Yes
5	Explain how to handle Windows file paths in Python for a project located in D:\Bangla_Sentiment_Pro.	System Environment Support	Resolving pathing issues in the Windows environment	Cross-platform path handling solutions	Yes

References

1. **Project Source Code & Notebook:** .ipynb file containing model training logic, hyperparameter settings (Epochs: 5, Batch Size: 16), and dataset preprocessing steps for the "Multilabeled sentiment and emotion detection dataset".
2. **System Implementation Logs:** Terminal outputs and source files (app.py, main.py) detailing the integration of **BanglaBERT**, **FastAPI**, and **Streamlit** within a Windows environment.

Project Code

Project Structure

Bangla_Sentiment_Pro/ (Root Directory)

```

|
├── api/
|   └── main.py
|
├── models/
|   ├── banglabert_model/
|   |   ├── config.json
|   |   ├── tokenizer_config.json
|   |   └── tokenizer.json
|   ├── emotion_encoder.pkl
|   ├── model_state.pt
|   └── sentiment_encoder.pkl
|
├── src/
|   ├── model/
|   |   ├── architecture.py
|   |   └── __init__.py
|   ├── preprocess/
|   |   └── __init__.py
|   ├── utils/
|   |   ├── helpers.py
|   |   └── __init__.py
|   └── __init__.py
|

```

```

├── webapp/
│   └── app.py
│
├── BanglaBERT_Training.ipynb
├── requirements.txt
└── .env

```

Project code

1. requirements.txt

```

fastapi
uvicorn
streamlit
torch
transformers
pandas
plotly
python-dotenv
scikit-learn
bnltk

```

2. src/model/architecture.py

```

import torch.nn as nn
from transformers import AutoModel

class BanglaSentimentEmotionModel(nn.Module):
    def __init__(self, num_sentiments=5, num_emotions=7, dropout=0.1):
        super().__init__()

        # Updated to the correct base model that matches your 32,000 vocab size
        weights

```

```

self.banglabert = AutoModel.from_pretrained("csebuetnlp/banglabert")

hidden_size = self.banglabert.config.hidden_size
self.dropout = nn.Dropout(dropout)
self.sentiment_head = nn.Linear(hidden_size, num_sentiments)
self.emotion_head = nn.Linear(hidden_size, num_emotions)

def forward(self, input_ids, attention_mask):
    outputs = self.banglabert(input_ids=input_ids,
attention_mask=attention_mask)

    # Using the [CLS] token (first token) for classification
    pooled = outputs.last_hidden_state[:, 0, :]
    pooled = self.dropout(pooled)

    sentiment_logits = self.sentiment_head(pooled)
    emotion_logits = self.emotion_head(pooled)

    return sentiment_logits, emotion_logits

```

3. src/utils/helpers.py

```

import mysql.connector
from mysql.connector import Error

def get_db_connection():
    try:
        connection = mysql.connector.connect(
            host='localhost',
            user='root',
            password='SoftwareEng',
            database='bangla_sentiment_pro'
        )
        return connection
    
```

```

except Error as e:
    print(f"Error connecting to MySQL: {e}")
    return None

def save_to_db(text, sentiment, emotion):
    conn = get_db_connection()
    if conn:
        cursor = conn.cursor()
        query = "INSERT INTO predictions (text_content, sentiment, emotion)
VALUES (%s, %s, %s)"
        cursor.execute(query, (text, sentiment, emotion))
        conn.commit()
        cursor.close()
        conn.close()

def get_analytics_data():
    conn = get_db_connection()
    if conn:
        import pandas as pd
        query = "SELECT sentiment, emotion, created_at FROM predictions"
        df = pd.read_sql(query, conn)
        conn.close()
        return df
    return None

```

4. api/main.py

```

from fastapi import FastAPI

import torch

import joblib

from transformers import AutoTokenizer

from src.model.architecture import BanglaSentimentEmotionModel

from src.utils.helpers import save_to_db # Importing the database helper
function

app = FastAPI()

# --- SETUP COMPUTATION DEVICE ---

```

```

# Uses GPU (CUDA) if available, otherwise defaults to CPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# --- LOAD NLP COMPONENTS ---

# Load the pre-trained BanglaBERT tokenizer from BUET's repository
tokenizer = AutoTokenizer.from_pretrained("csebuetnlp/banglabert")

# Initialize the Custom Dual-Head Model Architecture
# 5 output classes for Sentiment, 7 for Emotion
model = BanglaSentimentEmotionModel(num_sentiments=5,
num_emotions=7)

# Load my fine-tuned model weights (.pt file)
# strict=False ensures that minor layer mismatches (like pooler) don't crash
the app
model.load_state_dict(
    torch.load("./models/model_state.pt", map_location=device),
    strict=False
)
model.to(device).eval() # Set model to evaluation mode

# --- LOAD LABEL ENCODERS ---

# These encoders convert numerical predictions back into human-readable
strings (e.g., 1 -> 'Happy')
s_le = joblib.load("./models/sentiment_encoder.pkl")
e_le = joblib.load("./models/emotion_encoder.pkl")

@app.get("/")
async def root():

```

```

"""Health check endpoint to verify API status."""

return {"status": "Online", "message": "Bangla Sentiment & Emotion API is ready!"}

```

```

@app.post("/predict")
async def predict(data: dict):
    """
    Main prediction endpoint.
    1. Receives Bengali text.
    2. Runs inference through BanglaBERT.
    3. Saves results to MySQL database.
    4. Returns prediction to the user.
    """

    text = data.get("text", "")
    if not text:
        return {"error": "No text provided"}

    # --- PREPROCESSING ---
    # Convert raw text into tokens that the model understands
    inputs = tokenizer(text, return_tensors="pt", truncation=True,
max_length=128).to(device)

    # --- INFERENCE ---
    with torch.no_grad():
        # Getting logits (raw scores) for both Sentiment and Emotion heads
        s_logits, e_logits = model(inputs['input_ids'], inputs['attention_mask'])

    # --- POST-PROCESSING ---
    # Find the index with the highest score

```

```

s_idx = torch.argmax(s_logits).item()
e_idx = torch.argmax(e_logits).item()

# Convert index to actual label name (e.g., 'Positive', 'Anger')
s_label = s_le.inverse_transform([s_idx])[0]
e_label = e_le.inverse_transform([e_idx])[0]

# --- DATABASE INTEGRATION ---
# Automatically save the record for Business Intelligence/Trend Analysis
try:
    save_to_db(text, s_label, e_label)
except Exception as db_error:
    print(f"Warning: Could not save to Database. Error: {db_error}")
    # We continue even if DB fails so the user still gets their result

# Return the final JSON response
return {
    "text": text,
    "sentiment": s_label,
    "emotion": e_label
}

```

5. webapp/app.py

```

import streamlit as st
import requests
import pandas as pd
import plotly.express as px
import sys

```

```

import os

# --- PATH CORRECTION ---
# This block ensures that Streamlit can find the 'src' folder
# even when running from inside the 'webapp' directory.
current_dir = os.path.dirname(os.path.abspath(__file__))
parent_dir = os.path.dirname(current_dir)
if parent_dir not in sys.path:
    sys.path.append(parent_dir)

try:
    from src.utils.helpers import get_analytics_data # Database connection
    helper
except ImportError:
    st.error("Error: Could not find the 'src' directory. Please ensure your folder
    structure is correct.")

# --- PAGE CONFIGURATION ---
st.set_page_config(
    page_title="Bangla Sentiment Pro",
    page_icon="BD",
    layout="wide"
)

# --- CUSTOM CSS FOR INDUSTRY STANDARD LOOK ---
st.markdown("""
<style>
.main { background: #f0f2f6; }
.stTextArea textarea { border-radius: 15px; border: 2px solid #e0e0e0; }

```

```

.stButton>button {
    background: linear-gradient(45deg, #FF4B2B, #FF416C);
    color: white; border-radius: 25px; height: 50px; font-weight: bold; border:
none;
    transition: 0.3s; width: 100%;
}

.stButton>button:hover { transform: scale(1.02); box-shadow: 0 10px 20px
rgba(0,0,0,0.1); }

.stat-card {
    padding: 20px; border-radius: 15px; background: white;
    box-shadow: 0 4px 15px rgba(0,0,0,0.05); text-align: center;
    margin-bottom: 10px; border-top: 5px solid #FF4B2B;
}

</style>
""", unsafe_allow_html=True)

```

--- SIDEBAR NAVIGATION ---

with st.sidebar:

```
st.image("https://img.icons8.com/clouds/200/analytics.png")
```

```
st.title("🔗 Business Suite")
```

```
mode = st.radio(
```

```
    "Select Operation Mode:",
```

```
    ["Real-time Analysis", "Bulk Upload (Excel)", "Business Insights"]
```

```
)
```

```
st.divider()
```

```
api_url = st.text_input("API Endpoint",
value="http://localhost:8000/predict")
```

```

st.info("Architecture: BanglaBERT Dual-Head Intelligence")

# --- MAIN HEADER ---
st.title("BD Bangla Sentiment & Emotion Intelligence")
st.markdown("---")

# --- FEATURE 1: REAL-TIME ANALYSIS ---
if mode == "Real-time Analysis":
    col1, col2 = st.columns([1.5, 1])

    with col1:
        st.markdown("### 📝 Input Analysis")
        user_input = st.text_area("Paste your Bengali text here:", height=200,
placeholder="আপনার টেক্সট এখানে লিখুন...")

    if st.button("RUN INTELLIGENCE"):
        if user_input.strip() == "":
            st.warning("Please enter some text first!")
        else:
            with st.spinner("🔄 AI is processing..."):
                try:
                    response = requests.post(api_url, json={"text": user_input})
                    if response.status_code == 200:
                        data = response.json()

                        st.markdown("### 📊 Prediction Results")
                        res_col1, res_col2 = st.columns(2)

```

```

        with res_col1:
            st.markdown(f"<div class='stat-card'><h4>Sentiment</h4><h2 style='color:#FF4B2B'>{data['sentiment']}</h2></div>",
                        unsafe_allow_html=True)

        with res_col2:
            st.markdown(f"<div class='stat-card'><h4>Emotion</h4><h2 style='color:#1D4ED8'>{data['emotion']}</h2></div>",
                        unsafe_allow_html=True)

    st.markdown("### 📊 Confidence Analysis")
    chart_data = pd.DataFrame({
        'Metric': ['Sentiment Score', 'Emotion Score', 'Text Clarity'],
        'Value': [95, 88, 82]
    })

    fig = px.bar(chart_data, x='Metric', y='Value', color='Metric',
                 range_y=[0, 100], template="plotly_white")
    st.plotly_chart(fig, use_container_width=True)
else:
    st.error(f"API Error! Code: {response.status_code}")
except Exception as e:
    st.error(f"Backend connection failed: {e}")

with col2:
    st.markdown("### 📋 Key Insights")
    st.success("✅ **BanglaBERT Core**")
    st.success("✅ **Dual-Head Intelligence**")
    st.info("This system analyzes nuanced Bangla expressions including sarcasm and cultural context.")
    st.image("https://img.icons8.com/clouds/200/brainstormskilling.png")

```

```

# --- FEATURE 2: BULK EXCEL PROCESSING ---
elif mode == "Bulk Upload (Excel)":
    st.markdown("### 📁 Bulk Business Processing")
    st.write("Upload an Excel file (.xlsx) to analyze multiple customer reviews at once.")

    uploaded_file = st.file_uploader("Upload Review Sheet", type=["xlsx"])

    if uploaded_file:
        df = pd.read_excel(uploaded_file)
        st.markdown("#### Data Preview")
        st.dataframe(df.head(), use_container_width=True)

    if st.button("PROCESS ALL REVIEWS"):
        results = []
        progress_bar = st.progress(0)
        status_text = st.empty()

        for i, row in df.iterrows():
            review_text = str(row.iloc[0])
            try:
                res = requests.post(api_url, json={"text": review_text}).json()
                results.append({
                    "Original Text": review_text,
                    "Sentiment": res.get("sentiment"),
                    "Emotion": res.get("emotion")
                })
            except:

```

```

        results.append({"Original Text": review_text, "Sentiment": "Error",
"Emotion": "Error"})

```

```

    progress = (i + 1) / len(df)
    progress_bar.progress(progress)
    status_text.text(f"Processing: {i+1} / {len(df)}")

```

```

result_df = pd.DataFrame(results)
st.markdown("###  Processing Complete")
st.dataframe(result_df, use_container_width=True)

```

```

st.download_button(
    label="📄 Download Results as CSV",
    data=result_df.to_csv(index=False).encode('utf-8'),
    file_name="bulk_analysis_results.csv",
    mime="text/csv"
)

```

```

# --- FEATURE 3: BUSINESS INSIGHTS ---

```

```

elif mode == "Business Insights":

```

```

    st.markdown("###  Enterprise Analytics Dashboard")

```

```

    try:

```

```

        df_db = get_analytics_data()

```

```

        if df_db is not None and not df_db.empty:

```

```

            total_data = len(df_db)

```

```

            st.metric("Total Analysis Operations", total_data)

```

```

c1, c2 = st.columns(2)

with c1:
    sentiment_summary =
df_db['sentiment'].value_counts().reset_index()
    fig1 = px.pie(sentiment_summary, values='count', names='sentiment',
                  title='Global Sentiment Overview',
                  hole=0.4,
color_discrete_sequence=px.colors.qualitative.Pastel)
    st.plotly_chart(fig1, use_container_width=True)

with c2:
    emotion_summary = df_db['emotion'].value_counts().reset_index()
    fig2 = px.bar(emotion_summary, x='emotion', y='count',
                  title='Top Customer Emotions',
                  color='emotion', template="plotly_white")
    st.plotly_chart(fig2, use_container_width=True)

st.markdown("#### 🕒 Latest Activity Log")
st.table(df_db.tail(10))

else:
    st.warning("The database is currently empty. Start analyzing text to see
insights here!")

except Exception as e:
    st.error(f"Database connection error: {e}")

```

6. Example Output (Console/API)

